

DOTVERSE

Build Plan · Architecture · Execution Protocol

Architecture · Lessons Learned · GAP Analysis
GAP 2 → GAP 6 Detailed Build Plans
Pre-Code Protocol · Identity Contract · Failure Brainstorm

GAP 2 HIGH

GAP 3 HIGH

GAP 4 HIGH

GAP 5 MED

GAP 6 MED

May 2026

1 · System Architecture

Current State — May 2026

DotVerse Runtime Components

The system runs as four concurrent processes on Railway cloud infrastructure, communicating over PostgreSQL and Redis. The MetaTrader 5 EA polls the Flask API every 5 seconds and pushes account state every 5 seconds — making the EA the execution and data bridge between the cloud system and the live MT5 terminal.

Flask API	Python / Gunicorn	Signal engine · MT5 bridge · Telegram webhook · All endpoints	Railway — auto-deploy on git push
Frontend	Vanilla JS / HTML ~12,000 lines	Single-file SPA · Dashboard · Scanner · Calculator · Act tab	Served by Flask static
RQ Worker	Python / redis-queue	Background jobs: auto-scan (15 min) · watch job (60s) · daily summary	Railway — second service, same codebase
PostgreSQL	Railway add-on Port 46116	MT5Orders · SignalHistory · Positions · AutomationSettings · Watches	metro.proxy.rlwy.net sslmode=disable
Redis	Railway add-on Port 20577	OHLCV cache (5 min TTL) · Scanner signal cache · Dedup keys · RQ queue	metro.proxy.rlwy.net
MT5 EA	MQL5	Polls /api/mt5/pending every 5s · Pushes state every 5s · Sends level alerts on TP/SL hits	Runs on trader's MT5 terminal
Telegram Bot	Telegram API webhook	Receives signals · Execute button · Close/Breakeven/Reanalyze one-tap actions	Webhook registered on deploy

Identity Architecture — The Golden Rule

This contract governs every database write, every mt5_state lookup, and every query involving user ownership. It is non-negotiable and must be applied before writing any new function:

Context	user_id value	Examples
EA polls (/api/mt5/pending)	"default"	The EA has no session. It never authenticates as a specific user.
Telegram execute / auto-scan orders	"default"	_job_auto_scan and telegram_webhook create MT5Order(user_id="default")
Human user — frontend actions	str(session.get("user_id")) e.g. "42"	mt5_submit_order, automation_settings_save, add_watch all use session user_id
mt5_state dictionary key	EA pushes body.get("user_id", "default")	In practice always "default". Any state lookup must fallback: mt5_state.get(uid) or mt5_state.get("default", {})

Queries that serve the EA / background jobs	Must include "default" in filter	MT5Order.user_id.in_(["default"]) _get_automation_settings: prefer human row over default row
---	----------------------------------	--

2 · GAP 1 — Verification Against Original PDF Spec

Chapters 5–9

Before committing GAP 1 fixes, every feature in the original PDF spec is verified against what was actually implemented. Gaps still open are carried forward as named GAPS 2–6.

Ch 5 — Live Position Monitor	60s loop · macro event check · current profit pips · stop distance vs event volatility	PARTIAL	Signal invalidation + ATR expansion: DONE. Event-calendar trigger ("Close 50%") = GAP 2.
Ch 5	"Close 50% + Move to Breakeven" and "Keep Full Position" Telegram buttons triggered by event risk	GAP 2	Handlers exist (partial_close / ignore). Trigger logic in run_watch_job = NOT YET BUILT.
Ch 7 — SL Cascade	TP1 → SL to entry (breakeven). TP2 → SL to TP1 price. TP3 → SL to TP2 price.	DONE	mt5_level_alert Phase C. Bug S fixed: 3-path original lookup. Bug M fixed: fill_price fallback.
Ch 7	Cascade fires on ALL remaining open positions on that symbol (not just the hit position)	DONE	Iterates mt5_state for remaining tickets on same symbol. Queues one MODIFY per ticket.
Ch 7	Telegram notification sent at every cascade step with locked profit amount	DONE	send_telegram() fires with ladder_label on every cascade event.
Ch 8 — ATR Dynamic Trailing	Trail distance = ATR × multiplier. Recalculated every 60s. Modify queued if ATR changes >10%.	DONE	run_watch_job _trail_dist_cache 10% gate. Bug O fixed: adaptive precision. Bug Y fixed: mt5_state fallback.
Ch 8	PDF says default trail = 1.0× ATR. Conservative = 0.75× ATR.	GAP 6	Code default is trailing_atr_mult=2.0. PDF says 1.0. GAP 6 = change default to 1.0.
Ch 9 — Signal Invalidation	Confluence score drops below 50% → invalidation alert	DONE	run_watch_job 5a. Bug X fixed: user_id filter. Bug Y fixed: mt5_state fallback.
Ch 9	ATR doubles from entry → TP targets conservative → re-run analysis alert	DONE	run_watch_job 5b. ATR expansion 2× threshold. Telegram keyboard with Re-run and partial_close.
Ch 9	EMA fast crosses below EMA slow on a BUY trade → invalidation trigger	GAP 3	Not yet implemented. GAP 3 = add EMA cross detection to run_watch_job invalidation block.
Ch 9	Supertrend flips direction mid-trade → invalidation trigger	GAP 3	Not yet implemented. GAP 3 = add Supertrend flip detection.
Ch 9	Alert text: "Tighten stop to +X pips (above entry)" with specific pip recommendation	GAP 4	Current alert text is generic. GAP 4 = compute specific pip recommendation from current profit.

Ch 4 — Event Intelligence	Real-time news sentiment in signal card (positive / negative / neutral headlines)	GAP 5	_narrate_data_openai uses OpenAI for verdict. Finnhub news sentiment = GAP 5.
---------------------------	---	-------	---

3 · Lessons Learned — Root Cause Analysis

13 bugs · 3 patterns

GAP 1 produced 13 bugs requiring 16 code fixes across multiple adversarial review rounds. The bugs were not random. They cluster into three repeating root patterns that would have been prevented by the protocol defined in Chapter 4 of this document.

Pattern 1 — Identity Contract Violated (4 bugs: V, W, X, Y)

The system has two identity contexts: EA/background jobs use `user_id="default"`. Humans save settings under their real session `user_id` (e.g. "42"). This was architecturally known but never written down as a rule. Every function was written as if only one context existed.

Bug	Function	Symptom	Root Cause
V	mt5_get_state enrichment	TP2/TP3 always blank for Telegram-executed positions	Query filtered <code>user_id=="42"</code> but orders saved as "default"
W	_get_automation_settings	EA ignores all user trailing/breakeven settings	Always returned "default" row even when user customised under "42"
X	run_watch_job invalidation queries	Signal invalidation never fired for any live position	DB query filtered <code>user_id=="42"</code> , missed "default" Telegram orders
Y	run_watch_job mt5_state lookups	Trailing stop and invalidation never fired	<code>mt5_state.get("42")</code> returned <code>{}</code> — EA always pushes to "default"

Pattern 2 — Incomplete Feature Propagation (4 bugs: N, P, Q, R)

New fields added to one side of a data flow but not traced to every consumer. The code was written without first mapping all data producers and consumers.

Bug	Field Missing	Added Here	Missing From
N	tp2, tp3	MT5Order model	ScanAlert model · _record_scan_alert · Telegram execute path
P	entry_confluence, entry_atr	MT5Order model	ScanAlert · _record_scan_alert · _job_auto_scan · execute path
Q	level_hits	mt5_level_alert (sets it)	mt5_push_state (wiped it every 5s — preserved on read, not on write)
R	None guard on MT5Order	mt5_confirm_order	order.tp2 raised <code>AttributeError</code> when order lookup returned <code>None</code>

Pattern 3 — Domain Knowledge Not Researched Before Coding (2 bugs: S, U)

MT5-specific behaviour and forex conventions were assumed rather than verified. Both required five minutes of reading before writing the code.

Bug	Assumption	Reality
-----	------------	---------

S	res.deal == PositionGetTicket() in MT5	MT5 deal ticket (res.deal, returned by OrderSend) is DIFFERENT from position ticket (PositionGetTicket). The Phase C cascade queried mt5_ticket == deal_ticket — always None. Cascade silently never fired.
U	"JPY" in str(entry_price).upper() detects JPY pairs	entry_price is a float like 150.23. "JPY" is never in a float string. pip_size was always 0.0001 for JPY — lot size was 100x too small.

4 · Build Approach — Pre-Code Protocol

Applied to every GAP from this point forward

The following protocol replaces the previous approach of writing code first and finding bugs through adversarial review. Every GAP in this document was designed using this protocol before a single line of code was specified.

Step 1	Write the Data Flow Specification Before opening app.py, write: what data exists, where it comes from, what user_id context applies to every read and write, what fields are needed, what consumers exist downstream. Draw the complete data flow as a table, not prose.
Step 2	Apply the Identity Contract Explicitly decide for every new function: does this serve the EA ("default") or a human user (real user_id)? Write the decision in a comment at the function level. Never allow "I'll sort out user_id later".
Step 3	Map All Callers and Consumers (GitNexus) Run gitnexus_impact() on every function being modified. Report blast radius. If impact = HIGH or CRITICAL, the plan requires user review before proceeding.
Step 4	Write the Failure Brainstorm For every new block of logic, answer six questions: (1) data assumption gaps, (2) math edge cases, (3) empty/malformed inputs, (4) what user sees when wrong, (5) fast-clicker/race conditions, (6) cross-feature interactions. Each answer becomes either a tested criterion or an explicit "untested" entry.
Step 5	Write Success Criteria Before Code Define exactly what DONE looks like. "The alert fires" is not a criterion. "When position open > 4h and event in 48h and event avg move >= stop distance, Telegram receives one message within 60s with partial_close callback_data matching {ticket}" is a criterion.
Step 6	Complete Features — No Partial Propagation If a new field is added to model A, trace every consumer of model A and update them. If a new callback_data format is added, confirm the handler exists. Never submit a field that has no reader. Never submit a handler with no sender.
Step 7	One Function, One Commit, Two Gates CLAUDE.md rule: every commit is one logical change. Show the diff. Ask "Shall I commit?" Wait for yes. Then ask "Shall I push?" Wait for yes. Never bundle. Never push without explicit confirmation.

5 · GAP 2 — Live Position Event Monitor

HIGH PRIORITY · Chapter 5 of original PDF

PDF Chapter 5 specifies: when a HIGH/MEDIUM impact macro event is approaching within 48 hours and the event's expected volatility (historical average move) is $\geq$ the current stop distance, the system must alert the trader with specific numbers and one-tap execution buttons. This is the only GAP 1 feature from Chapter 5 that was not yet implemented.

Data Flow Specification

Data Item	Source	User ID context
Open positions (ticket, symbol, open_price, volume)	mt5_state.get(uid) or mt5_state.get("default", {})	"default" fallback
Current price of instrument	ind["price"] from run_watch_job indicator calc	N/A — market data
Position TP1 price (for % toward TP1)	MT5Order.tp where user_id.in_([uid, "default"])	Both
Macro events (name, time, impact, avg_move_pips)	_get_macro_context_inline(ticker, asset_type) — already called in run_watch_job	N/A — external API
Stop loss price	mt5_state position sl field, or MT5Order.sl	Both
New MT5Order (CLOSE / MODIFY)	Created in run_watch_job, saved to DB	"default"
Dedup key	Redis: "evt_alert:{ticket}:{YYYY-MM-DD-HH//4}" TTL=14400s	N/A

Trigger Condition

A position qualifies for an event alert when ALL of the following are true simultaneously within a single 60s watch cycle:

1	A HIGH or MEDIUM impact event exists for this position's instrument within 48 hours	Matched from _get_macro_context_inline events list
2	event["avg_move_pips"] $\geq$ stop_distance_pips (event volatility $\geq$ stop distance)	Prevents alerts for events too small to affect the stop
3	No event alert was sent for this ticket in the current 4-hour window	Redis dedup. If Redis is down, skip the alert (no spam risk from unavailability)
4	Position has not hit TP1 yet (no level_hits entry for this ticket)	No point alerting on a position that has already booked partial profit

Message Format (from PDF Chapter 5)

```
LIVE ALERT — {SYMBOL} {direction} · Ticket #{ticket}
Opened {open_time}. Currently at {current_price} — +{profit_pips} pips (+{pct_toward_tp1:.0f}% toward TP1).
{event_name} releases in {hours_away} hours ({event_time} UTC, {impact} IMPACT). {symbol} average move on {event_name}: {avg_move_pips} pips.
Your stop is at {sl_price} — {stop_distance_pips} pips below current price. The expected event volatility ({avg_move_pips} pips) {comparison} your stop distance.

Option A: Close 50% at +{profit_pips} pips. Move remaining stop to entry ({open_price}, breakeven).
```

Option B: Hold full position. Full risk if event surprises.

Telegram Keyboard

callback_data formats — both handlers already exist in telegram_webhook:

Button	callback_data	Handler	Action
Close 50% + Move to Breakeven	partial_close{{ticket}}{{symbol}}event_risk	EXISTING — partial_close handler	Queues CLOSE 50% + MODIFY sl=open_price in one transaction
Keep Full Position — I understand the risk	ignore{{ticket}}{{symbol}}event_risk	EXISTING — ignore handler	Dismisses keyboard only. No order queued.

Failure Brainstorm

avg_move_pips missing from event data: Some events have no historical avg move. If avg_move_pips is None or 0, skip the event (no comparison possible without baseline). Log: "[evt_monitor] {event}: no avg_move_pips — skipped".

stop_distance_pips = 0: Position sl may be None (market order without SL). Guard: if sl is None or sl == 0, skip this position entirely.

open_price unavailable: pos.get("open_price") or pos.get("price_open"). If both None, skip — cannot compute profit or breakeven.

TP1 not in DB for this position: MT5Order lookup may fail for very old orders. pct_toward_tp1 = None — omit from message rather than crashing.

Redis down: Cannot dedup. Skip entire evt_monitor block for this cycle. Do not send without dedup (spam risk).

Multiple events for same instrument: Take the soonest HIGH-impact event. Fall back to soonest MEDIUM. One alert per position per 4-hour window regardless.

Same position matches both invalidation AND event alert: Both can fire independently. They have separate dedup keys. This is correct — they are different warnings.

Telegram double-tap on button: Keyboard is removed by _tg_dismiss_keyboard after first tap. Second tap: stale callback, answerCallbackQuery returns error silently. No duplicate order.

Implementation Scope

One function modified: run_watch_job. ~40 lines added inside existing position loop, after the signal invalidation block. No new model. No new migration. No new endpoint. No new Telegram handler. callback_data formats already handled.

Blast radius: run_watch_job only. No shared CSS. No shared endpoints. MEDIUM risk.

6 · GAP 3 — Signal Invalidation: EMA Cross + Supertrend Flip

HIGH PRIORITY · Chapter 9 of original PDF

PDF Chapter 9 specifies two additional triggers for the signal invalidation alert beyond confluence drop: EMA fast crossing below EMA slow (on a BUY), and Supertrend flipping direction. The current implementation only monitors confluence percentage. These triggers detect structural market changes that confluence alone may not catch.

Data Flow Specification

Data Item	Source	Already Available?
EMA fast value (current bar)	ind["ema_fast"] — calculate_indicators output	YES — in run_watch_job ind dict
EMA slow value (current bar)	ind["ema_slow"]	YES
EMA fast value (previous bar)	calculate_indicators returns last-bar values only	NO — need to store prev bar value in watch_registry
Supertrend direction (current)	ind["supertrend_bull"] — bool in indicator dict	YES
Supertrend direction (previous)	Must be stored in watch_registry between ticks	NO — need to store prev supertrend_bull
Position direction (BUY or SELL)	MT5Order.order_type — queried by ticket	YES — already done in invalidation block
Dedup key (separate from confluence dedup)	Redis: "ema_cross:{ticket}:{date}" TTL=86400s	NEW

State Storage in watch_registry

Previous-bar values must survive between 60-second watch_job ticks. watch_registry[key] already stores last_check, last_price, last_signal. Two new fields added per watch entry:

```
watch_registry[key]["prev_ema_cross"] = "above" | "below" (EMA fast vs slow)
watch_registry[key]["prev_supertrend_bull"] = True | False
```

Trigger Logic

EMA Cross (BUY position)	prev_ema_cross == "above" AND ind["ema_fast"] < ind["ema_slow"] AND _stored.order_type == "BUY"	EMA fast was above EMA slow last tick. Now below. Cross happened this 60s window.
EMA Cross (SELL position)	prev_ema_cross == "below" AND ind["ema_fast"] > ind["ema_slow"] AND _stored.order_type == "SELL"	Mirror: EMA fast crossing above slow on a SELL = invalidation
Supertrend Flip (BUY)	prev_supertrend_bull == True AND ind["supertrend_bull"] == False	Supertrend was bullish. Now bearish. Flip happened this tick.
Supertrend Flip (SELL)	prev_supertrend_bull == False AND ind["supertrend_bull"] == True	Mirror: flip to bullish on a SELL = invalidation

Implementation Scope

Two locations modified: (1) run_watch_job watch_registry update block — store prev_ema_cross and prev_supertrend_bull after each tick. (2) Signal invalidation block — add EMA cross and Supertrend flip checks before the existing confluence check. No new model. No new migration. One new Redis dedup key pattern.

One commit per trigger type (EMA cross = one commit, Supertrend flip = one commit). Two commits total.

7 · GAP 4 — Invalidation Alert Diagnostic Detail

HIGH PRIORITY · Chapter 9 of original PDF

PDF Chapter 9 shows a specific alert format: "Tighten stop to +8 pips (above entry, locking partial profit) and wait for re-confirmation." The current implementation sends a generic recommendation. GAP 4 computes the specific pip value from actual position data.

Current vs Required Alert Text

Current (Generic)	Required (Specific — PDF Ch 9)
Consider moving your stop to breakeven or exiting to protect capital.	Tighten stop to +{safe_pips} pips (above entry {open_price}, locking {safe_pips * pip_value:.2f} profit). If confluence does not recover above 60% on the next 2 candles, consider closing the position to preserve profit.

Calculation: safe_pips

safe_pips = current profit in pips minus a 3-pip buffer (keeps stop above entry to lock at least breakeven). Capped at TP1 distance minus 10%.

```
profit_pips = abs(current_price - open_price) / pip_size
safe_pips = max(1, round(profit_pips - 3, 0)) # at least 1 pip above entry
```

New Telegram Keyboard for Invalidation Alert

The existing breakeven handler already moves SL to open_price. For GAP 4 a new callback_data format is needed for "tighten to +N pips":

```
tighten|{ticket}|{symbol}|{sl_price} - telegram_webhook handles similarly to breakeven
```

This IS a new callback_data prefix. A new elif block is needed in telegram_webhook. Only one new MT5Order(order_type="MODIFY", sl=float(sl_price), action="modify_sl"). This is the smallest handler possible — ~10 lines.

Implementation Scope

Two locations: (1) run_watch_job invalidation alert — replace generic text with computed specific text + new keyboard. (2) telegram_webhook — add "tighten" handler elif (~10 lines). One commit.

8 · GAP 5 — News Sentiment via Finnhub

MEDIUM PRIORITY · Chapter 4 of original PDF

PDF Chapter 4 specifies real-time news sentiment as one of three live data sources checked at every signal. Currently `_narrate_data_openai` receives news as part of the OpenAI prompt context but the Finnhub sentiment scoring (positive / negative / neutral count + headline list) is not surfaced separately on the signal card.

What Finnhub Provides

Finnhub `/news?category=general&token;={key}` and `/company-news?symbol={symbol}&from;={}&to;={}` return JSON arrays of articles with headline, datetime, source, sentiment (where available). DotVerse already has a Finnhub economic calendar integration. The same API key can query the news endpoint. No new API key required.

Signal Card Addition

Field	Computation	Display
Positive headlines	Count articles with sentiment > 0.2 in last 24h	Green number
Negative headlines	Count articles with sentiment < -0.2 in last 24h	Red number
Neutral headlines	Count articles where sentiment <= 0.2	Grey number
Sentiment verdict	net = positive - negative: +3 → BULLISH PRESS, -3 → NEGATIVE PRESS, else NEUTRAL	Amber tag on signal card
Top headline	Most recent article headline (truncated to 80 chars)	Italic below verdict

Failure Brainstorm

No Finnhub API key configured: Return None. `_narrate_data_openai` skips news section. Signal card shows no news widget. No crash.

Finnhub rate limit (60 calls/min free tier): Cache news response in Redis: `"news:{ticker}:{YYYY-MM-DD-HH}"` TTL=3600s. Signal for same ticker in same hour reuses cache.

Finnhub returns empty array: `sentiment_verdict` = "NEWS UNAVAILABLE". Card shows grey indicator.

Crypto tickers (BTC-USD): Finnhub news uses `symbol="BTCUSD"` not `"BTC-USD"`. Apply `_mt5_symbol()` mapping.

Sentiment field absent: Not all Finnhub articles include a sentiment score. Fallback: keyword-based scoring on headline text (bearish: "crash", "drop", "fear" / bullish: "surge", "rally", "all-time").

Implementation Scope

One new function: `_fetch_news_sentiment(ticker, asset_type) → dict`. Called inside `_get_macro_context_inline` (already called at signal time). Result added to `response_data["news_sentiment"]`. Frontend signal card: one new row in the event-context block.

Two commits: (1) backend `_fetch_news_sentiment` + API integration, (2) frontend signal card news row.

9 · GAP 6 — ATR Trailing Default 2.0x → 1.0x

MEDIUM PRIORITY · Chapter 8 of original PDF

PDF Chapter 8 states: "The trail distance is set to 1.0x current ATR... Conservative option: 0.75x ATR." The current default is `trailing_atr_mult=2.0`. At 2.0x ATR the trailing stop is twice as wide as the spec, meaning Position 10 gives back significantly more profit before closing. This is a spec deviation.

Impact Analysis

Setting	BTC ATR 480 pips	BTC ATR 890 pips (volatile)	Result
Current default: 2.0x	Trail: 960 pips	Trail: 1,780 pips	Position gives back 960–1780 pips before close. Significantly more than optimal.
PDF spec default: 1.0x	Trail: 480 pips	Trail: 890 pips	Position gives back 480–890 pips. Matches spec intent.
Conservative: 0.75x	Trail: 360 pips	Trail: 668 pips	Tighter — less room. Appropriate for quieter conditions.

Change Required

Three locations:

- AutomationSettings model: `trailing_atr_mult = Column(Float, default=1.0)`
- `_init_db` migration: `ALTER TABLE automation_settings ALTER COLUMN trailing_atr_mult SET DEFAULT 1.0`
- `_get_automation_settings` fallback: `"trailing_atr_mult": getattr(s, "trailing_atr_mult", 1.0) or 1.0`

Existing rows in the DB already have value 2.0 (set when first created). The migration does NOT update existing rows — only changes the default for new rows. Existing users who have customised will be unaffected.

One commit. No frontend change. No new endpoint. Blast radius: AutomationSettings defaults only.

10 · Build Sequence & Commit Plan

GAP 2 → 6 · Sequential execution

Each GAP is a separate Railway deploy. No GAP starts until the previous one is runtime-verified by the user in a live browser. No bundling.

0	GAP 1 fixes — commit + push	READY	1 commit 16 fixes	run_watch_job, mt5_level_alert, _get_automation_settings, _calc_auto_lot, mt5_get_state	git log confirms commit. Railway deploy shows healthy. User confirms live.
1	GAP 2: Event-triggered "Close 50%" alert	HIGH	1 commit	run_watch_job only (+~40 lines in position loop)	Force a matching scenario: watch EUR/USD with open position + CPI in 48h. Confirm Telegram fires.
2	GAP 3a: EMA cross invalidation trigger	HIGH	1 commit	run_watch_job — store prev_ema in watch_registry + check	Confirm watch_registry stores prev_ema_cross. Manual cross test with historical tick.
3	GAP 3b: Supertrend flip invalidation trigger	HIGH	1 commit	run_watch_job — store prev_supertrend_bull + check	Confirm prev_supertrend_bull stored. Supertrend flip test.
4	GAP 4: Invalidation alert diagnostic text + tighten button	HIGH	1 commit	run_watch_job alert text + telegram_webhook tighten handler (~10 lines)	Trigger invalidation alert. Confirm "tighten to +N pips" shows specific value. Tap button → MODIFY queued.
5	GAP 5: News sentiment (Finnhub)	MED	2 commits (backend + frontend)	_get_macro_context_inline, _fetch_news_sentiment (new), frontend signal card	Analyze BTC or EURUSD. Confirm news sentiment row appears on signal card.
6	GAP 6: ATR trailing default 2.0→1.0	MED	1 commit	AutomationSettings model + _init_db migration + _get_automation_settings	New installation gets 1.0× default. Existing user row unchanged at 2.0 (by design).

Non-Negotiable Rules For This Build Phase

- Every GAP starts with the Identity Contract check: which user_id context does this code serve?
- Every new field is traced to ALL consumers before the commit is made.
- Every new callback_data prefix has a handler in telegram_webhook before the commit.
- GitNexus gitnexus_impact() is run on every function being modified. HIGH/CRITICAL = user review required before proceeding.
- No GAP starts until the previous GAP is confirmed working in a live Railway browser session by the user.
- One logical change per commit. No bundling. Show diff → confirm commit → confirm push. Two gates.
- If a bug is found mid-build, STOP. Fix the bug. One commit for the bug fix. Resume the GAP on a new commit.
- Failure brainstorm is written before code for every new block of logic. Not after.

DotVerse Build Plan · Architecture & Execution Protocol · May 2026 · Confidential